

Нижe — подробное описание сущности **Plan** (План) в предлагаемой модели с учётом всех сценариев: виды плана, внутренние балансы во времени, правила миграции денег между планами, иерархия «родитель–дочерний» и всё необходимое поведение, чтобы выполнение всех ваших юзкейсов было корректным и реализуемым на практике.

## 1. Основная идея сущности Plan

Сущность **Plan** описывает **любой целевой контейнер** денег в вашей системе. Под «контейнером» понимается набор виртуальных средств, которые

- 1. **либо накапливаются** (накопительный план; «фонд», в который ежемесячно/разово закидываете деньги, чтобы к какой-то дате достичь цели);
- 2. **либо расходуются** (тратный план; «карманные», «абонемент», «бензин» и т. п., куда «стучатся» расходы);
- 3. **либо являются «буфером»** (VirtualPool — свободные средства, которые ещё не распределены ни в какие узкие планы);
- 4. **либо служат для «одноразовых» операций** (точечные покупки или разовые приходы).

Каждый Plan обладает набором полей, определяющих его тип, текущее состояние (балансы), а также правила, по которым деньги туда поступают и оттуда расходуются. За счёт единой сущности Plan мы получаем:

- минимальное дублирование схемы: все «фондовые», «накопительные» и «тратные» планы хранятся в одной таблице (с частью полей, актуальных только для некоторых типов);
- гибкость: при необходимости можно добавить ещё один «тип» и установить для него своё поведение;
- прозрачность для пользователя: одно и то же UI-компонент «карточка плана» достаточно универсален для всех разновидностей.

## 2. Полная структура таблицы Plan и назначение полей

|                                 |                                                                          |
|---------------------------------|--------------------------------------------------------------------------|
| Plan                            |                                                                          |
| ├ id                            | – PK (уникальный идентификатор плана)                                    |
| ├ user_id                       | – внешний ключ к пользователю (владелец плана)                           |
| ├ name                          | – строка: название плана (например, «Накопления на отпуск», «Карманные») |
| ├ type                          | – enum: {                                                                |
| │                               | "virtual_pool",     // «буфер» / «свободные                              |
| │ средства», без целевой задачи | "bucket_in",        // накопительный план                                |
| │ («фонд», «накопления»)        | "bucket_out",       // тратный план («карманные»,                        |
| │ «абонемент», «коммуналка»)    | "one_time_plan"     // одноразовый план (точечная                        |
| │ покупка или точечный приход)  | }                                                                        |
| └ parent_plan_id                | – FK → Plan.id (nullable). Если задан, значит этот                       |

план вложен в родительский.

- | — Полезно для группировки: «Фонд Дом → Ремонт → Окна».
- | — `priority` — integer (узкоприоритет распределения входящих средств).
- | — Чем меньше число — тем раньше «наполняется» этот план в алгоритме распределения.
- | — `current_virtual_balance` — decimal (сколько денег сейчас виртуально «лежит» в этом плане).
- | — `last_balance_update` — datetime (когда последний раз мы синхронизировали или пересчитали текущий баланс)
- | — `goal_amount` — decimal (nullable). Целевая сумма для накопительных планов (`bucket_in` и иногда `bucket_out`, если расход строго ограничен).
- | — `goal_date` — date (nullable). Дата, к которой нужно достичь `goal_amount`.
- | — `min_pessimistic_target` — decimal (nullable). Нижняя пессимистичная цель, ниже которой лучше не опускаться (в `bucket_in`).
- | — `target_milestones` — JSON (nullable). Список промежуточных точек { date: дата, amount: сумма }, чтобы отмечать на графиках.
- | — Пример: ``[{"date": "2025-09-01", "amount": 50000}, {"date": "2025-12-01", "amount": 100000}]``.
- | — `expected_weekly_amount` — decimal (nullable). Для `bucket_out`: «примерно столько уходит в неделю», чтобы генерировать виртуальные траты, если пользователь не хочет указывать строгую дату.
- | — `expected_monthly_amount` — decimal (nullable). Для `bucket_out`: «примерно столько уходит в месяц».
- | — `day_of_month` — integer (1-31, nullable). Если `bucket_out` является ежемесячным расходом с фиксированным днём списания (например, абонентская плата 5-го каждого месяца).
- | — `recurrence_type` — enum { "none", "weekly", "monthly", "annually" } (nullable).
- | — Определяет, как часто в рамках этого плана создаются виртуальные точечные `ExpectedTransaction`.
- | — `recurrence_detail` — JSON (nullable). Детали расписания, например:
  - для ``weekly``: ``{ "days_of_week": ["Mon", "Thu"] }``
  - для ``monthly``: ``{ "day_of_month": 5 }`` (или ``{ "last_day": true }``)
  - для ``annually``: ``{ "month": 12, "day": 31 }``
- | — `auto_spend_all` — boolean. Признак, что «все поступления (income) у этого пользователя автоматически тратятся на этот план» (например, пользователю не важно — он хочет, чтобы всё шло в один всёядный фонд).
- | — `is_archived` — boolean. Если true, план считается закрытым, и система его не учитывает в «активных» (скрывает из публичных списков и не генерирует виртуальные транзакции).
- | — `created_at` — datetime.
- | — `updated_at` — datetime.

## Разбор полей по типам плана

### 1. `type = "virtual_pool"`

- **Смысл:** «буфер» (те деньги, которые ещё не распределены ни в какие более узкие планы).
- **Используемые поля:**
  - `current_virtual_balance` (сколько свободных у пользователя денег);
  - `priority` (обычно ставится самым высоким, например, `priority = 1000`, чтобы в алгоритме распределения доходов VirtualPool наполнялся последним, после всех более приоритетных фондов/накоплений).
- **Неактуальные поля:** `goal_amount`, `goal_date`, `min_pessimistic_target`, `target_milestones`, `expected_*`, `recurrence_*`.
- **Поведение:**
  - Не генерирует виртуальных транзакций сам по себе (`recurrence_type = none`).
  - При входящем «приходе» (реальном Transaction или виртуальном ExpectedTransaction) деньги «скатываются» сначала к высшим в приоритете Plan, и **оставшееся** «залиточно» ложится в VirtualPool.

## 2. `type = "bucket_in"` (накопительный план; «фонд»)

- **Смысл:** «я хочу отложить деньги в этот фонд и накопить к определённой дате или просто держать там подушку».
- **Используемые поля:**
  - `current_virtual_balance` (текущий запас в этом фонде);
  - `goal_amount`, `goal_date` (если есть — значит требуется накопить именно эту сумму к этой дате);
  - `min_pessimistic_target` (если вы хотите отслеживать, чтобы пессимистично не упасть ниже X);
  - `target_milestones` (если есть ряд точек, по которым важно отмечать, например, «30 000 Р к концу июня, 50 000 Р к концу сентября»);
  - `priority` (решает, в каком порядке фонд «забирает» inbound средства);
  - `recurrence_type` / `recurrence_detail` – только если вы хотите, чтобы этот план сам «рекуррентно» (ежемесячно/еженедельно) генерировал «виртуальные транзакции» **поступления**. Это редкий сценарий: обычно virtual\_bucket\_in наполняется либо реальными транзакциями/ручными переводами, либо AllocationRule направляет на него часть доходов.
  - `auto_spend_all`: обычно для накопительного плана это false (мы не хотим, чтобы всё, что приходит, сразу туда уходило, только часть по правилам, а остальное – в другие планы или в VirtualPool).
- **Неактуальные поля:** `expected_weekly_amount`, `day_of_month` (если он не действует как регулярный таргет), но может использоваться для «автопополнения»: например, «каждый месяц откладываю 5000 в фонд» можно указать `recurrence_type = "monthly" + expected_monthly_amount = 5000 + day_of_month = 1`.
- **Поведение:**

1. **Приходы (реальные или виртуальные).** Если реальная транзакция `Transaction(direction = income)` поступает, в алгоритме распределения system смотрит:
  - есть ли для этой транзакции явно связанный `allocated_plan_id` (через `ExpectedTransaction` или прямой ручной перевод)? Если есть – кладёт именно туда.
  - иначе смотрит `AllocationRule` (если правило распределения определено) и, руководствуясь приоритетом, решает, какую долю положить в этот `bucket_in`.
  - часть, которая отводится именно этому фонду, увеличивает `current_virtual_balance`.
2. **Рекуррентные отчисления.** Если у плана стоят `recurrence_type = "monthly"` + `expected_monthly_amount = A` + `day_of_month = D`, то каждый месяц в дату D система создаёт внутри Plan «виртуальную положительную транзакцию» типа `ExpectedTransaction { date = D, amount = A, direction = income, source_type = "one_time_plan", source_id = Plan.id }`. Это нужно только если вы хотите, чтобы сам план автоматом генерировал поступления без участия `RecurringRule`.
3. **Расходы (выпуск из фонда).** Если вы хотите разрешить «их тратить», должны быть ручные или автоматические переводы из этого плана в другие планы (`ManualOverride` или `AllocationRule` для расходного типа). Иначе деньги «зависают» в фонде.
4. **Сценарии.** В режиме `pessimistic`, `base`, `optimistic` часть `Incoming` и часть `Outgoing` (через `Recurrence/AllocationRule`) меняется по коэффициентам (хранится в сопутствующих таблицах).

### 3. `type = "bucket_out"` (тратный план; «расходный»)

- **Смысл:** «я хочу тратить в этом направлении: продукты, бензин, карманные».
- **Используемые поля:**
  - `current_virtual_balance` (сколько там находится «зарезервировано» на будущие траты);
  - `expected_weekly_amount` и/или `expected_monthly_amount` (если вы хотите, чтобы система генерировала виртуальные расходы, даже когда вы не указываете точную дату).
  - `recurrence_type` / `recurrence_detail` указывают, **когда** именно система «генерирует» виртуальную транзакцию расхода. Пример: `recurrence_type = "weekly", recurrence_detail = {"days_of_week":["Sat","Sun"]}`, `expected_weekly_amount = 2000` → значит: каждую субботу и воскресенье система суммарно выделяет «виртуальную трату» в 2000 Р.
  - `day_of_month` (если ежемесячно, к примеру, «с 5-го каждого месяца уходит 3000Р»). Тогда `recurrence_type = "monthly", recurrence_detail = {"day_of_month":5}`, `expected_monthly_amount = 3000`.
  - `goal_amount` и `goal_date` могут использоваться, если вы хотите поставить какое-то ограничение: «я готов потратить не более 20 000 р. к 01.08». Тогда система считается

«заявленный лимит», и когда баланс «потребления» (отрицательный баланс) приближается к  $-20\,000$ , она будет вас предупреждать.

- **priority**: если несколько расходных планов нуждаются в финансировании (через трансферы из VirtualPool), важнее потратить в «Основные расходы» или в «Хобби»? Чем меньше **priority**, тем раньше распределится виртуальный бюджет.

○ **Неактуальные поля**: **goal\_amount**, **goal\_date** (если не нужен лимит).

○ **Поведение**:

### 1. Генерация виртуальных трат (ExpectedTransaction).

- Каждый «интервал» (неделя, месяц и т. п.) система создаёт «виртуальную расходную транзакцию» по этому плану: `ExpectedTransaction { date = D, amount = X, direction = "expense", allocated_plan_id = Plan.id }`.
- Дата  $D$  = текущий момент + интервал, рассчитывается по **recurrence\_type** и **recurrence\_detail**.
- Сумма  $X$  = **expected\_weekly\_amount** или **expected\_monthly\_amount** (в зависимости от типа).

### 2. Покрытие виртуальных трат «зарезервированными» средствами.

- Когда генерируется **ExpectedTransaction**, идёт проверка:

```
if Plan.current_virtual_balance >= X:
    Plan.current_virtual_balance -= X
else:
    // либо уходим в минус, либо берём из VirtualPool согласно
    // правилам
    if overdraft_allowed (настраивается в AllocationRule для
    расходных):
        Plan.current_virtual_balance -= X // уходим в
        отрицательный баланс
    else:
        // пытаемся взять разницу из VirtualPool:
        delta = X - Plan.current_virtual_balance
        Plan.current_virtual_balance = 0
        VirtualPool.current_virtual_balance -= delta // при
        этом update balance
        // если и в VirtualPool нет — система может показать
        предупреждение о нехватке
```

- Получается, что «зарезервированная» сумма либо списывается из самого Plan, либо, если его недостаточно, вытягивается из VirtualPool.

### 3. Когда реальный расход (Transaction) происходит и связан с этим планом (**related\_expected\_id = id виртуальной записи** или пользователь «связал транзакцию с Plan вручную»), мы «материализуем» виртуальную трату:

- `ExpectedTransaction.is_consumed = true`

- Создаётся Allocation: `Allocation { transaction_id = real_txn.id, plan_id = Plan.id, amount = real_txn.amount }`.
- Если оказалась «переплата» или «недоплата», появляется ручное перераспределение через `ManualOverride`.

#### 4. `type = "one_time_plan"` (одноразовый план)

- **Смысл:** «разовая трата» (покупка) или «разовый приход» (подарок/продажа что-то).
- **Используемые поля:**
  - `goal_amount` и `goal_date` вкпе с `current_virtual_balance` (на хранении «зарезервированных» средств до даты расхода).
  - `recurrence_type = none` (никаких регулярных операций).
  - `day_of_month` не нужен; вместо него есть единственное «плановое» поле `goal_date`.
  - `expected_*` не нужны (просто точечная «виртуальная транзакция» генерируется на дату `goal_date`).
- **Неактуальные поля:** `expected_weekly_amount`, `recurrence_detail`, `min_pessimistic_target` (если нет смысла строить пессимистичные прогнозы по точечной трате).
- **Поведение:**
  1. При создании `Plan(type="one_time_plan", goal_date = D, goal_amount = A, direction = "expense" или "income")` автоматически запускается генерация **одной** виртуальной транзакции:

```
ExpectedTransaction {
  date = D,
  amount = A,
  direction = (если это «покупка»: "expense", если
«поступление»: "income"),
  source_type = "one_time_plan",
  source_id = Plan.id,
  allocated_plan_id = Plan.id,
  scenario = base,
  is_consumed = false
}
```

2. До даты D деньги «зарезервированы» в этом плане: мы можем показать `Plan.current_virtual_balance += A` сразу (чтобы «деньги «лежали» в ожидании»); либо (более типично) ждать фактического прихода и не изменять `current_virtual_balance` до тех пор, пока пользователь явно «не подтвердит» транзакцию.
3. На дату D, когда пользователь «потратил» A (или когда `Transaction` с `related_expected_id` «закрыл» эту виртуальную точку), система сделает:
  - `ExpectedTransaction.is_consumed = true`.

- Сгенерирует реальный **Transaction** (если нужно) или привяжет существующий:

```
Transaction {
  date = D (или фактическая дата подтверждения),
  amount = A,
  direction = "expense" (или "income"),
  related_expected_id = ExpectedTransaction.id,
  category_id = (по желанию),
  ...
}
```

- Сразу создаст **Allocation { transaction\_id = Transaction.id, plan\_id = Plan.id, amount = A }**.
- **Plan.current\_virtual\_balance** сдвинется: если мы забирали заранее, он уйдёт в 0; если мы держали **current\_virtual\_balance=0** до факта, то при расходе он уйдёт в минус, и, согласно AllocationRule, часть может лечь на VirtualPool (или будет ошибка «нехватка средств»).

### 3. Балансы Plan во времени

У **Plan** есть два «основных» баланса:

#### 1. **current\_virtual\_balance**

- Отражает **независимо от сценария** (base/optimistic/pessimistic) реальную «окончательную точку» виртуальных денег **на текущий момент времени**.
- Обновляется при:
  1. Генерации виртуальных транзакций (ExpectedTransaction) → при «автоматической» генерации «периодических» трат/приходов по **recurrence\_type/recurrence\_detail**.
  2. Приходе реальных транзакций, когда они закрывают виртуальные (**ExpectedTransaction.is\_consumed = true**) → Allocation списывает или добавляет **amount** к **current\_virtual\_balance**.
  3. Ручных трансферов (ManualOverride) → прямое изменение **current\_virtual\_balance**.
  4. Cron-задачи, которая пересчитывает баланс по всем невыполненным виртуальным точкам.
- Всегда **текущее**: отображает состояние планов «по факту» (с учётом всего, что уже произошло даже в виртуализации, но до факта).

#### 2. **forecasted\_balance(date, scenario)** (неявно, генерируется через PlanBalanceSnapshot)

- Это **прогнозный баланс** на произвольную дату `date` в будущем (или прошлом) **для конкретного сценария** (`scenario = "base" | "optimistic" | "pessimistic"`).
- Вычисляется либо «on the fly» (агрегация виртуальных и фактических транзакций, учитывая коэффициенты), либо берётся из `PlanBalanceSnapshot` (если есть кэш).
- При расчёте участвуют:
  1. Все **фактические** `Allocation` (через реальные транзакции) до `now` (или до `date` при ретро-анализе).
  2. Все **виртуальные** `ExpectedTransaction` с `date' <= date`, с учётом `scenario` (т. е. `amount * scenario_factor`).
  3. Перемещения через `ManualOverride` с `date' <= date` (только те, у которых `scenario =` выбранный сценарий).
  4. Возможно, «коррекция» по `min_pessimistic_target` (в пессимистичном сценарии).
- Результатом получается «линия во времени» для каждого `Plan.id` и `scenario.id`:

```
PlanBalanceSnapshot { plan_id, scenario_id, date, balance_amount }
```

- Эти записи используются при построении графиков (Вьюхи 3, 8, 9 и т. п.).

Таким образом, **во времени** у `Plan` есть:

- **одно точечное «сейчас»**: `current_virtual_balance`, отражающее уже учтённые (виртуальные и реальные) события до момента `now`;
- **временная линия** (относящаяся к сценарию) → `PlanBalanceSnapshot` (кэш), которая позволяет мгновенно доставать «сколько будет» в `Plan` через N дней/месяцев.

## 4. Алгоритм «как деньги мигрируют» между `Plan`

Ниже приведён принцип единообразного «ориентировочного» алгоритма, который описывает, **что происходит с поступившей суммой** (реальной или виртуальной) и **как она распределяется** между планами.

### 4.1. Три вида «источников» денег

#### 1. Виртуальная транзакция (`ExpectedTransaction`):

- Сгенерирована автоматически в рамках `recurrence_type` или «одноразовым» `Plan` (`one_time_plan`).
- Поле `allocated_plan_id` обычно указывает на `Plan`, в который эти деньги должны «упасть» (например, virtual расход падает в `bucket_out`, virtual приход — в `bucket_in` или `VirtualPool`).

#### 2. Реальная транзакция (`Transaction`):

- Деньги уже физически пришли/списались (счёта пользователя).

- Если она **привязана** к `ExpectedTransaction (related_expected_id)`, значит «закрывает» виртуальную транзакцию и «подбирает» именно её.
- Если **не привязана**, система сама должна решить, куда распределить входящие (для дохода) либо откуда списать (для расхода), согласно `AllocationRule` и текущим `current_virtual_balance` Plan.

### 3. Ручной перевод (`ManualOverride`):

- Пользователь в UI говорит «хочу разово переместить сколько-то денег из Plan A в Plan B» (микро-режим).
- Система обновляет `Plan.current_virtual_balance` обоих Plan и, опционально, генерирует реальную транзакцию, если `transaction_effective = true`.
- Если это чисто виртуальная корректировка (`transaction_effective = false`), это **никак не** затрагивает реальный банк-счет — это просто перекидка «виртуальных» денег между Plan.

## 4.2. Распределение дохода (реального или виртуального)

Предположим, что в систему поступил **доход** `IncomingAmount = X` (либо виртуальный, либо реальный):

### 1. Если есть `related_expected_id` → значит заранее было решено, куда эти деньги пойдут:

- Из `ExpectedTransaction{allocated_plan_id}` сразу добавляем `X` к `Plan.current_virtual_balance` этого Plan.
- Если `Plan.type = "bucket_in"`, то это логично → увеличиваем баланс.
- Если `Plan.type = "virtual_pool"`, то это «жёсткий» `VirtualPool` → кладём весь `X` в `VirtualPool`.
- Если `Plan.type = "bucket_out"` или `one_time_plan` с `direction = income`, обрабатываем как «точечный приход»: кладём `X` в этот Plan и, возможно, дальше автоматически раздаём (через `AllocationRule`) в другие планы; редко, но поддерживается.

### 2. Если `related_expected_id = NULL` (т.е. real `Transaction` не был заранее привязан ни к какому Plan):

1. Берём конфигурацию `AllocationRule` для `source_type = "income"`.
2. Рассчитываем `priority_list` (список `{ plan_id:Pi, percent:Pi_% }` в порядке приоритета или без него; если не задан приоритет, сортируем по `Plan.priority` — чем меньше, тем выше возможность).
3. Проходимся по каждому плану из этого списка:

```
remaining = X
for (plan_id, percent) in priority_list:
    target_amount = X * percent    // либо сначала вычитаем
    фиксированные минимальные суммы, потом проценты
    if remaining <= 0: break
    if Plan(plan_id).type == "bucket_in":
        take = min(target_amount, remaining)
        Plan(plan_id).current_virtual_balance += take
```

```

        remaining -= take
    else if Plan(plan_id).type == "virtual_pool":
        // если доход «непривязанный» и правило говорит «положи остатки
        // в руч, а не в виртуальный пул»,
        // то достаточно просто break и перенести всё вот так:
        Plan(plan_id).current_virtual_balance += remaining
        remaining = 0
    else {
        // если перечислили bucket_out или one_time_plan в правило
        «куда класть доход»
        // – такое бывает редко, но поддерживаем:
        take = min(target_amount, remaining)
        Plan(plan_id).current_virtual_balance += take
        remaining -= take
    }
endfor
if remaining > 0:
    // если накоплены остатки, кладём их в VirtualPool (если в
    // приоритете он указан дальше),
    // либо просто игнорируем (но лучше положить в VirtualPool)
    Plan(VirtualPool.id).current_virtual_balance += remaining

```

4. Одновременно создаём **реальные** Allocation-записи: `Allocation { transaction_id, plan_id, amount = take }`.
5. Если остаток `remaining` не 0, но VirtualPool не позволяет (системно запрещено полагаться на VirtualPool), то либо возникает предупреждение «Избыток доходов некуда положить», либо весь избыток идёт в последний план, как «превышение».

### 4.3. Распределение расхода (реального или виртуального)

Предположим, что система должна «списать» **расход** `X`:

1. Если есть `related_expected_id` → значит заранее было известно, что этот расход уйдёт из заданного Plan `allocated_plan_id`.
  - `Plan.current_virtual_balance` уменьшается на `X`.
  - Если после списания баланс оказался отрицательным (`<0`), тогда действует правило `overdraft_allowed` (у него три варианта реализации):
    1. **`overdraft_allowed = true`** → баланс просто ушёл в минус; приложение показывает: «У плана «X» отрицательный баланс (–500 Р)». То есть разрешается «переступить» лимит и «копать в долги».
    2. **`overdraft_allowed = false`** → сначала пытаемся списать всё из Plan; если `Plan.current_virtual_balance < X`, то берём разницу из VirtualPool (если там есть).
      - Пусть `balance = Plan.current_virtual_balance`, `delta = X - balance`. Тогда:

```
Plan.current_virtual_balance = 0
VirtualPool.current_virtual_balance -= delta
```

- Если и в VirtualPool недостаток, то либо остаток расхода блокируется (ошибка «недостаточно средств»), либо мы уходим в глобальный минус VirtualPool (и плановая система сигнализирует: «недостаточно денег, ожидайте зарплату»).

3. **overdraft\_allowed = "only-if-parent"** (опциональное правило): позволяет уходить в минус только «дочерним» планам, при этом списание сначала идёт из дочернего, потом из родительского и так далее вверх по цепочке (описано ниже).

2. Если **related\_expected\_id = NULL** → значит у нас «спонтанный» расход без привязки к виртуальной точке (например, вы купили кофе, а система не успела сгенерировать виртуальный расход в «Траты на кофе»). Тогда:

1. Находим **список приоритетов** для расходов: смотрим **AllocationRule** для **source\_type = "expense"**. Возможно, там указан список «Plan1 (bucket\_out): 40 %, Plan2 (bucket\_out): 60 %».

2. Если правило не задано, **пытаемся автоматически подобрать Plan**, у которых

- **Plan.type = "bucket\_out"**
- **Plan.current\_virtual\_balance > 0**
- упорядочены по **priority ASC**.

3. Распределяем X по этим планам по их **current\_virtual\_balance** до тех пор, пока не «закроем» весь X:

```
remaining = X
for plan in all_active_bucket_out_plan ORDER BY priority:
    if remaining <= 0: break
    if plan.current_virtual_balance >= remaining:
        plan.current_virtual_balance -= remaining
        Allocation { transaction_id, plan_id=plan.id, amount =
remaining }
        remaining = 0
    else:
        Allocation { transaction_id, plan_id=plan.id, amount =
plan.current_virtual_balance }
        remaining -= plan.current_virtual_balance
        plan.current_virtual_balance = 0
endfor
if remaining > 0:
    // недостаточно «зарезервированных» средств; пытаемся списать
остаток из VirtualPool:
    if VirtualPool.current_virtual_balance >= remaining:
        VirtualPool.current_virtual_balance -= remaining
        Allocation { transaction_id, plan_id = VirtualPool.id, amount =
remaining }
        remaining = 0
    else:
```

```
// Нехватка средств → уходим в минус VirtualPool (или ошибка)
VirtualPool.current_virtual_balance -= remaining
Allocation { transaction_id, plan_id = VirtualPool.id, amount =
remaining }
remaining = 0
```

4. Если есть `overdraft_allowed` у некоторых Plan, то при `plan.current_virtual_balance = 0` и `overdraft_allowed = true` мы можем «уходить в минус» этому Plan и закрывать остаток расхода, не запуская VirtualPool.
5. Запись Allocation сохраняет всю «цепочку», чтобы легче было в истории понять, откуда именно списались деньги.

#### 4.4. Иерархия parent-child и «наследование» баланса

Поле `parent_plan_id` позволяет вам связывать Plan в **дерево** (любая глубина). Это используется, чтобы:

1. **Агрегировать балансы:** если у родительского плана есть несколько дочерних, его «реальный» (агрегированный) баланс = сумма балансов дочерних + свой собственный (если у родителя тоже когда-то спускали/кладали средства).
2. **Перенаправлять перерасход (overdraft) «вверх»:**
  - Часто бывает, что вы хотите потратить из «Хобби → Покупки игр», но денег там нет, тогда можно снимать из «Хобби → Общий карман» (родитель), а затем уже из VirtualPool.
  - При `overdraft_allowed = "only-if-parent"` для Plan дочернего уровня:

```
if child.current_virtual_balance < X:
    delta = X - child.current_virtual_balance
    child.current_virtual_balance = 0
    parent = Plan(parent_plan_id)
    if parent.current_virtual_balance >= delta:
        parent.current_virtual_balance -= delta
    else:
        // либо parent тоже уходит в минус,
        // либо забираем дальше из VirtualPool
```

#### 3. Групповая статистика и представления:

- В вьюхе «Сколько сейчас денег в Родительском Плани» отображается сумма по всем «детям».
- При «прогнозе» агрегированный баланс родителя строится как сумма прогнозных балансов детей.

#### 4. Планирование «большой цели через мелкие»:

- Например, Plan A = «Ремонт» (целевого amount = 200 000) имеет две дочерние: «Пол – 50 000» и «Краска – 30 000». Тогда полный прогноз на «Ремонт» = прогноз «Пол» + прогноз «Краска» + любое «собственное» (если в родителе есть `expected_monthly_amount`).

Чтобы **уменьшить дублирование**, мы храним непосредственно в **каждом Plan** только его `current_virtual_balance`. А **агрегированный** баланс родителя получаем **при запросе** (или через кэш в `PlanBalanceSnapshot`):

```
function getAggregatedBalance(plan_id):
    plan = SELECT * FROM Plan WHERE id = plan_id
    if no children:
        return plan.current_virtual_balance
    else:
        total = plan.current_virtual_balance
        for child in SELECT * FROM Plan WHERE parent_plan_id = plan_id:
            total += getAggregatedBalance(child.id)
        return total
```

Однако на практике **рекурсивные SQL-запросы** (CTE) или матричная таблица «Closure Table» могут использоваться для эффективного агрегационного запроса, чтобы не нагружать приложение.

## 5. Сценарии (optimistic / base / pessimistic) и как Plan учитывает коэффициенты

Чтобы учесть три «вала» прогноза, каждому Plan и связанным сущностям нужно:

1. **Хранить `scenario`** (либо в `ExpectedTransaction`, либо в отдельной таблице `Scenario`).
2. **Указывать, каким образом коэффициент применяется** для разных типов операций:
  - Для `recurrence_type` и `recurrence_detail` в Plan:
    - В поле `recurrence_amount` (можно завести явное поле `recurrence_amount` вместо раздачи `expected_weekly_amount` + `expected_monthly_amount`), добавить структуру:

```
{
  "base": 10000,
  "optimistic": 12000,
  "pessimistic": 8000
}
```

Вместо «числа» — JSON, где по ключу `scenario` лежит соответствующее значение. Тогда при расчёте виртуальных транзакций:

```
X = plan.recurrence_amount[current_scenario]
```

- Для **AllocationRule**:

- Вместо списка { `plan_id`, `percent` } просто храним JSON вида:

```
{
  "base": [ { "plan_id":10, "percent":0.5 }, { "plan_id":5,
    "percent":0.5 } ],
  "optimistic": [ ... ],
  "pessimistic": [ ... ]
}
```

- При распределении дохода или расхода берётся нужный список по выбранному `current_scenario`.

Когда пользователь в UI переключает сценарий (optimistic/base/pessimistic), вся логика «генерации виртуальных транзакций» и «распределения средств» выполняется с учётом этого сценария, а кэш `PlanBalanceSnapshot` перезагружается для выбранного сценария.

## 6. Как Plan «работает» шаг за шагом в типовом жизненном цикле

### 6.1. Создание нового Plan

#### 1. Пользователь запрашивает «Новый план» (карточка «Добавить план»).

- Он выбирает:
  - Название.
  - Тип: «Накопить» / «Потратить» / «Разовый».
  - Если «Накопить»: предлагает ввести `goal_amount` и (опционально) `goal_date`.
    - Также даётся возможность сразу указать `recurrence_type = monthly` и `recurrence_amount = <число>`, если вы хотите «вписать» регулярные переводы.
  - Если «Потратить»: предлагает указать «Разовый расход» (`one_time_plan`) или «Повторяющийся расход» (`bucket_out + recurrence`).
    - Для «Повторяющийся» запрашивает `recurrence_type`, `recurrence_detail` (день недели или день месяца) и `expected_amount`.
  - Если «Разовый приход/разовый расход»: задать `goal_date` (когда придёт или потратится) и сумму.
- Можно (по умолчанию) не указывать `parent_plan_id` — вложение задаётся опционально.
- `priority` по умолчанию ставится последним (например, 1000) и пользователь может его поменять вручную, если нужен более высокий приоритет.

- `min_pessimistic_target` и `target_milestones` пользователь укажет позже через «дополнительные настройки» (UI не требует их заполнения сразу).

## 2. Система сохраняет запись в `Plan`.

- `current_virtual_balance = 0` (обычно).
- `is_archived = false`.
- `created_at, updated_at = now`.

## 3. Если `Plan.type = "one_time_plan"` (разовый), система сразу создаёт 1 запись

`ExpectedTransaction` с:

```
{ date = goal_date,
  amount = goal_amount,
  direction = (expense или income),
  source_type = "one_time_plan",
  source_id = Plan.id,
  allocated_plan_id = Plan.id,
  scenario = текущий,
  is_consumed = false
}
```

и, **по желанию**, сразу делает `Plan.current_virtual_balance += goal_amount` (чтобы «зарезервировать» сумму).

- Если вы не хотите, чтобы виртуальный баланс изменился до факта, можно отложить обновление `current_virtual_balance` до тех пор, пока факт-приход/расход не произойдёт. Тогда `Plan` служит «лишь ярлыком» до момента D, и на дату D система либо:
  1. Исключительно генерирует `Transaction` / `Allocation`, минуя виртуальный баланс,
  2. Либо расплачивается по связке `ExpectedTransaction → Transaction`.

## 4. Если `Plan.recurrence_type ≠ "none"` (повторяющийся), система не создаёт сразу «тонну»

`ExpectedTransaction`, а лишь «привязывает» правило:

- На стороне БД сохраняются: `recurrence_type`, `recurrence_detail`, `expected_<period>_amount`.
- Фоновая задача (cron/job) либо «лениво при запросе» генерирует «виртуальные» `ExpectedTransaction` на каждую «следующую дату появления» (на основе `last_balance_update/now`) до заданного горизонта.
- Таким образом, у вас не будет «40392» будущих записей — лишь столько, сколько активно «нужно» (например, несколько месяцев вперед).

## 6.2. Ежедневная или периодическая работа «генератора виртуальных транзакций»

### 1. **Фоновая задача** (cron или при заходе пользователя на вкладку «Прогноз») запускается примерно так:

- Берёт `Plan` с `recurrence_type ≠ "none"` и `is_archived = false`.

- Для каждого планирует даты «с прошлого раза» (`last_balance_update`) **до ближайшей даты, когда нужны виртуальные события** (обычно генерируют на 2–3 периода вперёд, чтобы UI мог рисовать графики без задержек).
- Для каждого нового периода (дата D) формируется `ExpectedTransaction { date = D, amount = plan.expected_period_amount_for_current_scenario, direction = (income если bucket_in, expense если bucket_out), source_type="plan_recurrence", source_id = plan.id, allocated_plan_id = plan.id, scenario=current_scenario, is_consumed=false }`.
- Обновляется `last_balance_update = now`.

2. **Если при генерации** обнаруживается, что `Plan.current_virtual_balance` нужно скорректировать (например, когда генерируется виртуальный расход `bucket_out`, мы сразу же «резервируем» деньги, вычитая `expected_amount` из `current_virtual_balance` или из `VirtualPool`, если `Plan` пуст), тогда:

```
if plan.current_virtual_balance >= X:
    plan.current_virtual_balance -= X
else:
    delta = X - plan.current_virtual_balance
    plan.current_virtual_balance = 0
    VirtualPool.current_virtual_balance =
    VirtualPool.current_virtual_balance - delta
```

После этого запись `Plan` сохраняется.

3. **Если генерируется виртуальный прихода (`bucket_in`)**, то мы тоже можем сразу «кладём» X в `Plan.current_virtual_balance` (но зачастую приходы реализуют через `AllocationRule` из `VirtualPool` → `plan`).

### 6.3. Появление реальной транзакции (Transaction)

Когда возникает **Transaction** (пользователь ввёл вручную или пришёл импортом из банка):

1. **Если `Transaction.related_expected_id` задан**, значит мы «закрываем» виртуальный прогноз:

- `et = ExpectedTransaction.where(id = related_expected_id)`
- `et.is_consumed = true`
- В зависимости от `et.direction` (income/expense) мы:
  - Для **income**:
    - `Plan(allocated_plan_id).current_virtual_balance += et.amount` (если заранее не клали);
    - Если: `Plan.type = "one_time_plan"` → мы после этого «автоматически» архивируем `Plan` или помечаем его «выполненным».
  - Для **expense**:

- `Plan(allocated_plan_id).current_virtual_balance -= et.amount` (если заранее не списывали);
  - Если ушли в минус и `overdraft_allowed = false` → пытаемся забрать остаток из VirtualPool.
- Записываем `Allocation { transaction_id, plan_id = et.allocated_plan_id, amount = et.amount }`.

2. Если `Transaction.related_expected_id = NULL`, значит:

- Смотрим `AllocationRule` для `source_type = Transaction.direction`.
- Если правило есть → «размазываем» по Plan, как описано выше (раздел 4.2 и 4.3).
- Если правила нет → «ищем» активные Plan с `type = bucket_out` (для расхода) или `type = bucket_in` (для прихода) и разбираемся по приоритету, пытаюсь закрыть `Transaction.amount`.
- Для каждого кусочка (`plan_id, amount`) создаём `Allocation`.
- Обновляем `Plan.current_virtual_balance` каждого вовлечённого Plan (уменьшение при расходе, увеличение при приходе).

3. Если при любом шаге у Plan будет отрицательный `current_virtual_balance` и `overdraft_allowed = true`, это считается нормальным — приложение просто показывает, что «этот план ушёл в минус» (и отражает это графически).

- Если же это нежелательно (`overdraft_allowed = false`), то сброс лишних денег идёт через VirtualPool (или через `parent_plan`, если `parent-child overdraft` разрешён).

## 6.4. Ручной перевод (ManualOverride)

1. Пользователь кликает «Перевести деньги» (микро-режим), заполняет минимум полей:

- `from_plan_id, to_plan_id, amount, date = сейчас` (или можно выбрать будущее).
- (Опционально) `transaction_effective = true/false`. Если `true`, значит сразу генерится «реальный» Transaction, иначе — только виртуальное изменение.
- По умолчанию `scenario = base`.

2. Система

- Проверяет, что `Plan(from).current_virtual_balance >= amount` (или, если `overdraft_allowed` у `from = true`, то позволяем уходить в минус).
- `Plan(from).current_virtual_balance -= amount`
- `Plan(to).current_virtual_balance += amount`
- Создает запись `ManualOverride { ... }`.
- Если `transaction_effective = true`, также создает реальный Transaction `{ date, direction = ("expense" если from_plan отдаёт деньги или "income" если to_plan получает... но чаще это будет Expense), amount, related_manual_override_id = new_manual.id }`.

- И дальше, используя тот же алгоритм распределения, в том real **Transaction** создаются **Allocation** (как будто это был «непривязанный» Transaction).

### 3. Эффект

- Карточки Plan сразу обновляются, показывают изменённые балансы.
- При построении прогноза ManualOverride учитывается (в кэше PlanBalanceSnapshot), поэтому «через месяц» будет учтён этот перевод при расчётах.

## 7. Потенциальные «ловушки» и способы их решения

При таком полном описании **появляется несколько точек, где нужно быть внимательнее**, чтобы избежать несоответствий и багажу.

### 7.1. Несовпадение суммы «виртуальных» и «реальных» транзакций

- **Сценарий:** пользователь создаёт виртуальную трату (**ExpectedTransaction**) «каждый понедельник 100 ₽ на кофе», но потом пять понедельников подряд он фактически купил кофе за 120 ₽, 90 ₽, 110 ₽, 100 ₽, 95 ₽.
- **Проблема:** система накопила 5 виртуальных ExpectedTransaction ( $100 \text{ ₽} \times 5 = 500 \text{ ₽}$ ), а real Transaction суммарно списали ( $120+90+110+100+95=515 \text{ ₽}$ ).
- **Решение:**
  1. Система должна дать пользователю инструмент «отработать расхождения». Например, при закрытии каждого реального Transaction искать ближайший виртуальный ExpectedTransaction по дате, ставить `is_consumed = true`, и если real > виртуальный (например,  $120 \text{ ₽} > 100 \text{ ₽}$ ), то записывать **Allocation { plan\_id, amount = 100 }** + **Allocation { VirtualPool.id, amount = 20 }** (или «перераспределение» по допустимым правилам).
  2. Если real < виртуальный (например,  $90 \text{ ₽} < 100 \text{ ₽}$ ), то virtual остаток (10 ₽) остаётся в плановом резерве (или переводится обратно в VirtualPool).
  3. UI должен позволять «просмотреть все расхождения» и корректировать: «я хочу, чтобы вместо {100, 100, 100, 100, 100} реально списалось {120, 90, ...} — правильно?»

Таким образом, **Plan сам по себе служит только «контейнером», а точная обработка расхождений ложится на логику распределения (Allocation+ManualOverride)**, а не на сам Plan. Plan просто хранит текущий резерв.

### 7.2. Опасность «двойного учёта» при ручных переводах и виртуальных транзакциях

- **Сценарий:** пользователь сделал **ManualOverride** «перевёл 500 ₽ из VirtualPool в План А», но через несколько дней вот как-то ещё **RecurringRule** снова «подкинул» 500 ₽ в План А — получились «лишние» 500.
- **Проблема:** за полтора часа вы «дважды отдали» плану те же средства, хотя реально второй раз «кладывать» ничего не нужно.
- **Решение:**

1. При **ManualOverride** на уровне Plan ставить флаг: «сегодня уже в План А внесено 500Р вручную; не ревьёт рекуррент уже в этот же период».
2. Либо **сразу создавать ExpectedTransaction** из ManualOverride таким же образом, как RecurringRule делает, и не генерировать больше обычных виртуальных, пока «ручка» не «проглотит» этот период.
3. Или при генерации виртуальных записывать «pseudo-bound» на ManualOverride.id, чтобы понять, что для этой даты Enough.

В любом случае, чтобы **не произошло «двойного счёта»**, нужно иметь механизм «Deduplication» виртуальных источников, который сверяется с ручными переопределениями.

### 7.3. Проблема «Когда генерировать виртуальные транзакции»: ленивый vs агрессивный подход

- Если генерировать на **фиксированный календарный отрезок** («все даты до конца года»), то таблица **ExpectedTransaction** быстро станет огромной (365 записей × количество Plan).
- Если генерировать «лениво, только по запросу» (когда строим график, то сразу «подгоняем» виртуальные до нужной даты), то при высоких нагрузках/многопользователях возможны задержки, когда десятки тысяч операций должны сработать «на лету».

#### Решение:

1. Сочетание «ленивого» и «кэшированного» подхода:

- Храним в Plan поле **next\_expected\_date** (следующая дата, для которой ещё не создан ExpectedTransaction).
- Когда пользователь запрашивает прогноз до даты D, система перебирает:

```
while plan.next_expected_date <= D:
    create ExpectedTransaction for plan.next_expected_date
    update plan.next_expected_date = plan.next_expected_date + interval
```

- Таким образом, генерируем только «до нужного горизонта», а не «до бесконечности».

2. Периодически (cron) «догоняем» — на сервере есть nightly-job, который генерирует виртуальные транзакции для всех Plan на ближайшие 30 дней (или месяц), чтобы UI, похоже, «никогда не видел пустоту».
3. **Кэшируем** всё, что можно: **PlanBalanceSnapshot** для ускорения, позволяет быстро отображать графики.

### 7.4. Сценарии «оптимистичный/пессимистичный» при пересечении parent-child

- **Ситуация:** у вас есть ParentPlan «Ремонт» (min\_pessimistic\_target = 150 000 к 01.10) и два childPlan:
  1. «Окна» нужно 60 000, min\_pessimistic = 50 000;
  2. «Полы» нужно 90 000, min\_pessimistic = 80 000.

- **Проблема:** что значит «пессимистично» для Parent? Это  $\max(50\,000 + 80\,000) = 130\,000$ ? Или надо ещё учесть, что на Parent ещё лежит «свой» CurrentBalance?
- **Решение:**
  - **Определить чёткую бизнес-логику:**
    1. `Parent.min_pessimistic_target = (sum of child.min_pessimistic_target) + parent.own_min_pessimistic` (если есть).
    2. При генерации виртуальных транзакций и при прогнозе «факт + virtual до date D» строим сначала по children, а потом суммируем с parent.
  - Если Parent имеет собственный `recurrence_type` (например, он тоже «сам по себе» генерирует 10 000 Р в месяц на «ремонт»), это тоже берётся в расчёт.
  - Явно документировать в UI: «минимальная пессимистичная цель Parent = сумма минимальных целей детей + собственного»; если нужно иное поведение, придётся всё равно писать 1–2 строки кода, а не пытаться сделать «умное» вычисление автоматически.

## 7.5. Использование `goal_amount` и `min_pessimistic_target` одновременно

- **Сценарий:** план «Отпуск» с `goal_amount = 100 000 Р` к 01.08 и `min_pessimistic_target = 70 000 Р`.
- **Проблема:** в пессимистичном сценарии (зарплата меньше), цель может достигаться лишь на 80 000, а всё равно пользователю интересно, что «да, вы хоть 70 000 к 01.08 наверняка наберёте».
- **Решение:**
  1. При построении прогноза «base» или «optimistic» мы выводим обычную линию «Progress (AccumulatedForecast) → 100 000 к 01.08».
  2. При «pessimistic» рядом строим пункт «70 000 к 01.08» (из `min_pessimistic_target`), и для прогноза «сколько реально будет» используем коэффициенты `scenario_factor` для `RecurrenceRule` и `AllocationRule`.
  3. Если `AccumulatedForecast(pessimistic) < 70 000`, то красим график «ниже отталкивающей линии» и сигналим об опасности.

Таким образом, **Plan** в коде должен уметь возвращать две характеристики:

- `plan.getForecastedBalance(date, scenario)`
- `plan.getMinimumPessimisticTarget(date)` (которое может быть либо константой `min_pessimistic_target` с учётом `target_milestones`, либо суммой дочерних, либо динамически пересчитанной).

## 8. Полный перечень поведения Plan по типу

### 8.1. `type = "virtual_pool"`

- **Баланс:**

- **current\_virtual\_balance** = все входящие (реальные и виртуальные income), после распределения по другим Plan.
- **Recurrence**: нет (всегда none).
- **Приоритет**: низкий (всегда ставим priority = max, например, 1000).
- **Использование**:
  - все остатки/дефициты уходят/приходят сюда, если нет более приоритетных Plan.
- **Примеры**: «Остаток», «Наличные».

## 8.2. type = "bucket\_in" (накопительный план)

- **Баланс**:
  - **current\_virtual\_balance** = сумма реально + виртуально отложенных средств.
- **Recurrence**: опционально (если хотим «самоподкладывающийся» приход, например, «каждый месяц + 10 000 Р»).
- **Priority**: 1...N (чем меньше, тем раньше забирает доход).
- **Goal**: **goal\_amount** + **goal\_date** (если не заданы — план «открытый», можно просто держать там деньги без жёстких сроков).
- **Min-pessimist**: **min\_pessimistic\_target**.
- **Milestones**: из **target\_milestones**.
- **При поступлении доходов (вирт или реал)**:
  1. Если «привязан к Expected» → кладем в этот plan конкретную сумму.
  2. Иначе, согласно **AllocationRule**, кладем % или фиксированные суммы.
  3. Если исходящий «Максимум» превышен, часть идёт в следующий plan (по priority) или в VirtualPool.
- **При расходе**:
  - Редко, но может быть, что bucket\_in «отдаёт» бабки (через ManualOverride или AllocationRule).

## 8.3. type = "bucket\_out" (тратный план)

- **Баланс**:
  - **current\_virtual\_balance** = сумма «зарезервированных» средств, которые «отложены» под будущие траты.
- **Recurrence**: обычно «да» (каждую неделю/месяц мы «зарезервируем» X рублей).
  - При генерации виртуальной транзакции (тратная) план пытается списать **expected\_amount** → либо уходит в минус, либо забирает часть из VirtualPool.

- **Priority:** 1...N (сравнивается только с другими bucket\_out, когда у нас несколько «одновременных» расходов).
- **Goal:** служит лимитом, если задан (goal\_amount, goal\_date).
- **Min-pessimist:** откровенно редко используется (скорее всего, не важно).
- **При расходе:**
  - Если real Transaction приходит, он либо «закрывает» виртуальный расход через related\_expected\_id, либо, если анонимный, система сама «берёт» X из этого plan → Allocation.
  - Если current\_virtual\_balance < X и overdraft\_allowed = false, забираем остаток X из VirtualPool (или parent\_plan).

#### 8.4. type = "one\_time\_plan" (одноразовый)

- **Баланс:**
  - Пока до даты D средств нет (можно current\_virtual\_balance = 0 до факта).
  - На дату D создаём ExpectedTransaction(amount=goal\_amount, direction=expense/income).
  - Либо сразу кладём goal\_amount в current\_virtual\_balance (если хотим «заранее показать резерв»).
- **Recurrence:** none.
- **При поступлении/списании:**
  1. «Один раз» создаётся виртуальный приход или расход.
  2. Когда real Transaction приходит и связывается с этим ExpectedTransaction, мы делаем Allocation → Plan.current\_virtual\_balance смещается в зависимости от direction.
  3. По факту Plan отмечается «выполненным» (либо is\_archived = true).

## 9. Резюмирующее описание поведения Plan «в стиле README»

Сущность Plan совместима со всеми сценариями вашего приложения. Ниже кратко, как она реагирует на основные события:

### 1. Создание Plan:

- type = "virtual\_pool":
  - Сразу ready, current\_virtual\_balance = 0.
  - Приходы (виртуальные или реальные) после распределения по остальным планам падают сюда.
- type = "bucket\_in":
  - Может иметь goal\_amount, goal\_date, min\_pessimistic\_target, target\_milestones.
  - Может иметь recurrence\_type (если нужен регулярный «вклад»).
  - При создании current\_virtual\_balance = 0 (или сразу +initial\_amount, если

есть одноразовый приход).

- `type = "bucket_out"`:
  - Может иметь `expected_weekly_amount / expected_monthly_amount + recurrence_type`.
  - При регулярном «top-up» (генерации виртуального расхода) пытается сразу резервировать деньги (вычитать из себя или из `VirtualPool`).

- `type = "one_time_plan"`:
  - При создании задаётся `goal_amount` и `goal_date`; сразу же порождается `ExpectedTransaction`.
  - `current_virtual_balance` остаётся 0 (или `goal_amount`, если хотите мгновенного резерва, но лучше создавать резерв при фактическом приходе).

## 2. Генерация виртуальных транзакций (`ExpectedTransaction`):

- По `Plan.recurrence_type`: каждый период (неделя/месяц/год) создаётся `ExpectedTransaction`.
  - Если `Plan.type = bucket_in` → virtual приход.
  - Если `Plan.type = bucket_out` → virtual расход (и сразу списываем / резервируем сумму).
- Для `one_time_plan` создаётся ровно один `ExpectedTransaction` в дату `goal_date`.

## 3. Появление реальных `Transaction`:

- Если связан с `ExpectedTransaction` (`related_expected_id != NULL`):
  - Закрываем виртуальную транзакцию (`is_consumed = true`).
  - Создаём `Allocation` (`transaction_id` → `plan_id`).
  - Увеличиваем/уменьшаем `Plan.current_virtual_balance` (если не делали заранее).
  - Если получилось «расхождение» (`real > virt` или `real < virt`), создаём `ManualOverride` или автоматически перекладываем разницу в `VirtualPool`.
- Если «спонтанный» real `Transaction` (`related_expected_id = NULL`):
  - Смотрим `AllocationRule source_type = income/expense`.
  - Распределяем сумму по `Plan` (`bucket_in` или `bucket_out`) в порядке приоритетов + процентов + минимальных чисел.
  - Для тех `Plan`, которые забирают/кладут, меняем `current_virtual_balance`.
  - Записываем `Allocation` для каждого (`transaction_id`, `plan_id`, `amount`).

## 4. Ручные корректировки (`ManualOverride`):

- Пользователь сам «переливает» X из одного `Plan` в другой.
- `Plan(from).current_virtual_balance -= X`; `Plan(to).current_virtual_balance += X`.
- Сохраняется запись `ManualOverride` для истории и прогноза.
- Если нужно, вы генерируете real `Transaction`, связывая его с `ManualOverride`, и тогда создаётся `Allocation`.

## 5. Иерархия (`parent_plan_id`):

- Агрегированный баланс родителя = его собственный `current_virtual_balance` + суммы `current_virtual_balance` всех детей (рекурсивно).
- Если дочерний `plan` уходит в минус и `overdraft_allowed = "only-if-parent"`, то остатки забираются из родительского плана (если там есть) → далее из `VirtualPool`.

## 6. Сценарии (`optimistic / base / pessimistic`):

- Поля `plan.recurrence_amount` (или `expected_*`),

`plan.AllocationRule.scenario_modifiers` хранят значения под каждый сценарий.

- При построении прогноза (`PlanBalanceSnapshot`) все `virtual` события и распределения берутся с учётом текущего сценария (коэффициент, % и т. п.).

#### 7. Хранение балансов:

- `Plan.current_virtual_balance` → «точка сейчас» (фактическое/виртуальное состояние).
- `PlanBalanceSnapshot` → «линия во времени» (дата, баланс) для выбранного сценария; заполняется фоном или «лениво при запросе».

#### 8. Проверка консистентности:

- Система регулярно сравнивает:
  - $\sum \text{InventorySnapshot.total\_actual\_balance} == \sum \text{real Transaction (income - expense)} == \sum \text{Plan.current\_virtual\_balance}$  (по активным `Plan`).
- Если расхождение, показывает «на дату D потеря/нахождение X ₽» и позволяет погуглить, где именно «упали» средства.

#### 9. Надёжность и производительность:

- Генерация виртуальных транзакций †«лениво», с ограничением горизонта (например, +3 месяца вперед).
- Кэширование прогнозов (`PlanBalanceSnapshot`) для быстрой отрисовки графиков.
- Иерархическая агрегация через рекурсивные CTE (или `Closure Table`) для получения балансов «групп».

#### 10. UX-ориентированность:

- При создании `Plan` минимальный набор полей: «название», «тип (накопить/потратить/разовый)», «сумма/дата/частота».
- Подробные «минимальные» и «мило» поля (пессимистичная цель, `milestone`) во «вторичном экране» настроек.
- Drag-n-drop панель `Plan` для «где украсть/позаимствовать» при ручных переводах.
- Переключатель сценариев в шапке, пересчёт прогноза за секунду.
- Экран «Расхождения» (`Inventory vs Transactions vs Plans`).

В результате у вас **есть** одноузловая, но вместительная сущность `Plan`, способная «вести себя» по-разному в зависимости от поля ``type``, контролировать накопления, трату, точки «резерва» и условные «одноразовые» события, поддерживать иерархические структуры, учитывать несколько сценариев и давать пользователю **максимально простой** интерфейс («название + тип + сумма + дата/частота»), не требующий каждый раз «заполнять 15 полей».

Если в процессе реализации вы столкнётесь с **конкретными неясностями**, вот несколько рекомендаций:

#### 1. Как избежать «сковородочного дублирования» виртуальных транзакций:

- Ввести в `Plan` поле ``next_expected_date`` и генерировать виртуальные ``ExpectedTransaction`` только **последовательно** на каждый период, пока ``next_expected_date` ≤ horizon_date``.
- Если пользователь вручную изменил перевод на этот же период (`ManualOverride`), отмечаем, что «виртуальную транзакцию на этот период пропускаем».

#### 2. Как аккуратно считать «агрегированный баланс» `parent_plan`:

- Либо рекурсивный CTE (``WITH RECURSIVE``) в SQL, либо «`Closure Table`» (таблица, в которой явно хранятся все связи рода-родитель → `descendant`, каждый раз при

изменении parent-child перезаписывать).

- Если планов не очень много, допускается вычислять рекурсивно в коде, но лучше «SQL-специфически» сделать рекурсивный запрос

```
```sql
WITH RECURSIVE tree AS (
  SELECT id, current_virtual_balance FROM Plan WHERE id = :target_id
  UNION ALL
  SELECT p.id, p.current_virtual_balance
     FROM Plan p
     JOIN tree t ON p.parent_plan_id = t.id
)
SELECT SUM(current_virtual_balance) FROM tree;
```
```

### 3. **Как хранить «сценарные коэффициенты»:**

- Для Plan сделать JSON-поле ``recurrence_amounts`` = { "optimistic":12000, "base":10000, "pessimistic":8000 } вместо отдельного ``expected_weekly_amount``.
- Иначе придётся дублировать Plan для каждого сценария → тяжело.
- Аналогично в AllocationRule и RecurringRule: JSON на три сценария → удобно переключать, достаточно простой код.

### 4. **Что делать, если пользователю нужно «разовый перевод спросонок»?**

- Используем ``ManualOverride`` { date=today, from=PlanA, to=PlanB, amount }.
- UI: два клика – «выбрать план → ввести сумму → OK». Заведомо чуть чуть «хуже», чем «заполнить большой form с 15 полями», но зато **всего 2 поля**.

Если **всё это реализовать**, у вас получится:

- **Полнофункциональная** система планирования доходов и расходов,
- **Пользовательски удобная** (нет «лишних» галочек/полей),
- **Расширяемая** (приходят новые сценарии, типы планов – легко добавляем),
- **Производительная** (кэш прогнозов, ленивые виртуальные транзакции, рекурсивные СТЕ для агрегатов),
- **Консистентная** (три «слоя» всегда можно сверить: виртуальный, транзакционный, инвентаризационный).

Это именно та **единичная сущность Plan** с её поведением, которая «работает» для **всех описанных сценариев** и при этом остаётся **достаточно простой** с точки зрения юзабилити и реализации.